Ganpat University

Department of Computer Science

M.SC(CA & IT) SEM – 1 Practical Documentation

Subject : MERN Stack Development

Academic Year : 2026–2027

Submitted by :

Name : **Prince Prajapati**

Enrolment Number : **26084231038**

Index of Practicals

Practical No.	Practical Title	Status
01	Build a React application using Functional Components, JSX and Component Reusability concepts.	Completed
02	Develop a React application implementing Props, State Management and Event Handling.	Completed
03	Create a Student Management System using React Forms, Controlled Components and Validation.	Completed
04	Develop a Single Page Application (SPA) using React Router for navigation between multiple pages.	Completed
05	Create a backend server using Express.js implementing routing and CRUD operation.	Completed
06	Build RESTful APIs for Student Records Management using Express.js and Postman testing.	Completed
07	Develop an Employee Management API integrating Express.js with MongoDB Atlas.	Completed
08	Create a backend server using Express.js implementing routing, middleware and request handling.	Completed
09	Create a file upload module using Multer and store file metadata in MongoDB.	Completed
10	Develop a complete MERN application integrating React Frontend, Express Backend and MongoDB Database.	Completed
11	Create a Blog Management System implementing CRUD Operations, Authentication and Role-Based Access Control.	Completed
12	Develop a Real-Time Chat Application using Socket.IO, React and Node.js.	Completed
13	Mini Project: Design and develop a full-stack industry-oriented MERN application (Document Management System).	Completed

Practical – 01 : Build a React application using Functional Components, JSX and Component Reusability concepts.

1. Commands :

```
npm create vite@latest MERN_01
cd MERN_01
npm install
npm run dev
```

2. Project structure :

```
MERN_01/
|-- src/
|   |-- components/
|   |   |-- CourseCard.jsx
|   |   |-- Button.jsx
|   |-- App.jsx
|   |-- main.jsx
|-- package.json
```

3. Source Code :

- **src/components/Button.jsx :**

```jsx
import React from 'react';

export default function Button({ children, variant = 'primary', onClick }) {
  return (
    <button className={`btn btn-${variant}`} onClick={onClick}>
      {children}
    </button>
  );
}
```

- **src/components/CourseCard.jsx :**

```jsx
import React from 'react';
import Button from './Button';

export default function CourseCard({ title, instructor, duration, level, onEnroll }) {
  return (
    <div className="custom-card">
      <span className="badge">{level}</span>
      <h3>{title}</h3>
      <p>Instructor: {instructor} • Duration: {duration}</p>
      <Button variant="primary" onClick={() => onEnroll(title)}>Enroll Now</Button>
    </div>
  );
}
```

- **src/App.jsx :**

```jsx
import React from 'react';
import CourseCard from './components/CourseCard';

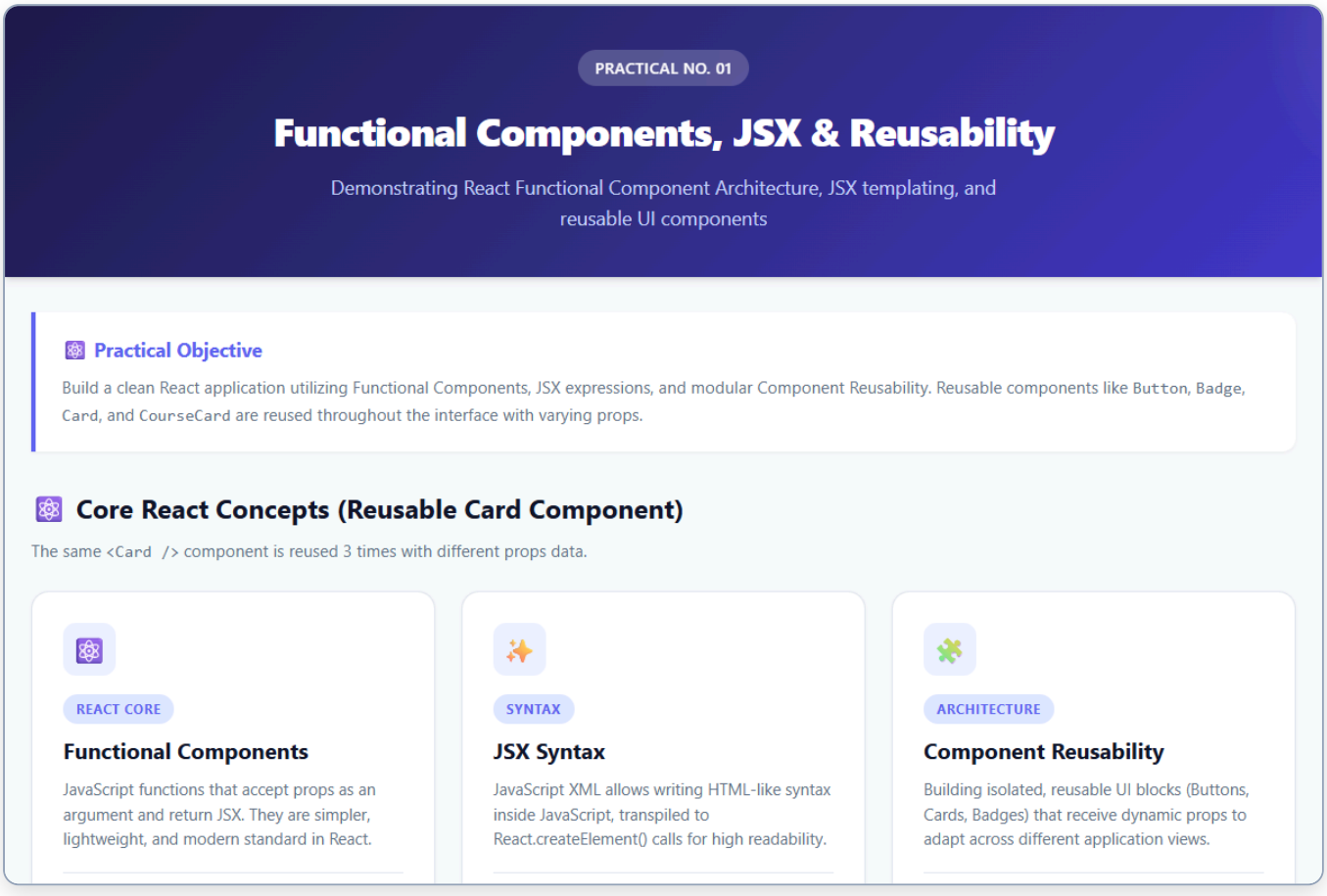
export default function App() {
  const courses = [
    { id: 1, title: 'Modern React with JSX', instructor: 'Dr. John Doe', duration: '6 Weeks', level:
'Beginner' },
```

```
    { id: 2, title: 'Backend Mastery with Express', instructor: 'Prof. Jane Smith', duration: '8
Weeks', level: 'Intermediate' },
    { id: 3, title: 'Full Stack MERN Architecture', instructor: 'Alex Johnson', duration: '12 Weeks',
level: 'Advanced' }
  ];

  return (
    <div className="app-container">
      <header className="header">
        <h1>Practical 01: Functional Components, JSX & Reusability</h1>
        <p>Demonstrating React Functional Architecture & Component Reuse</p>
      </header>
      <div className="grid-3">
        {courses.map(c => (
          <CourseCard key={c.id} {...c} onEnroll={(t) => alert(`Enrolled in: ${t}`)} />
        ))}
      </div>
    </div>
  );
}
```

4. Output :

The application renders functional components utilizing JSX expressions, passes props dynamically to reusable card and button components, and manages click interactions.



Practical – 02 : Develop a React Application implementing Props, State Management and Event Handling.

1. Commands :

```
npm create vite@latest MERN_02
cd MERN_02
npm install
npm run dev
```

2. Project structure :

```
MERN_02/
|-- src/
|   |-- components/
|   |   |-- CounterSection.jsx
|   |   |-- TaskManager.jsx
|   |-- App.jsx
|   |-- main.jsx
|-- package.json
```

3. Source Code :

- **src/components/CounterSection.jsx :**

```jsx
import React, { useState } from 'react';

export default function CounterSection() {
  const [count, setCount] = useState(0);
  const [step, setStep] = useState(1);

  return (
    <div className="card">
      <h3>Counter (State & Click Events)</h3>
      <div className="counter-display">{count}</div>
      <input type="number" value={step} onChange={(e) => setStep(Number(e.target.value) || 1)} />
      <button onClick={() => setCount(count + step)}>+{step}</button>
      <button onClick={() => setCount(Math.max(0, count - step))}>−{step}</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

- **src/components/TaskManager.jsx :**

```jsx
import React, { useState } from 'react';

export default function TaskManager() {
  const [tasks, setTasks] = useState([
    { id: 1, text: 'Learn React Props & State', completed: true },
    { id: 2, text: 'Master Synthetic Events', completed: false }
  ]);
  const [input, setInput] = useState('');

  const addTask = (e) => {
    e.preventDefault();
    if (!input.trim()) return;
    setTasks([...tasks, { id: Date.now(), text: input.trim(), completed: false }]);
    setInput('');
  };
```

```
  return (
    <div className="card">
      <h3>Task Manager (Array State & Callbacks)</h3>
      <form onSubmit={addTask}>
        <input value={input} onChange={(e) => setInput(e.target.value)} placeholder="New task..." />
        <button type="submit">Add Task</button>
      </form>
      {tasks.map(t => (
        <div key={t.id} onClick={() => setTasks(tasks.map(x => x.id === t.id ? { ...x, completed:
!x.completed } : x))}>
          <span style={{ textDecoration: t.completed ? 'line-through' : 'none' }}>{t.text}</span>
        </div>
      ))}
    </div>
  );
}
```

• **src/App.jsx :**

```
import React from 'react';
import CounterSection from './components/CounterSection';
import TaskManager from './components/TaskManager';

export default function App() {
  return (
    <div className="app-container">
      <header className="header">
        <h1>Practical 02: Props, State Management & Event Handling</h1>
        <p>Demonstrating useState Hook, Form Events & Uni-directional Data Flow</p>
      </header>
      <div className="grid-2">
        <CounterSection />
        <TaskManager />
      </div>
    </div>
  );
}
```

4. Output :

The application demonstrates interactive state updates using the useState hook, numeric counter arithmetic, parent-child prop callbacks, and form submission events.

PRACTICAL NO. 02

Props, State Management & Event Handling

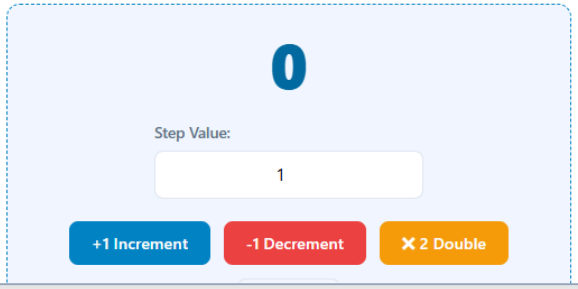
Practical implementation of Uni-directional Data Flow, useState Hook, and React Synthetic Event Handlers

Practical Objective

Develop a comprehensive React application demonstrating Props passing between parent-child components, local State Management with the `useState` hook, and handling diverse React Synthetic Events (`onClick`, `onChange`, `onSubmit`, `onKeyDown`, `onMouseMove`).

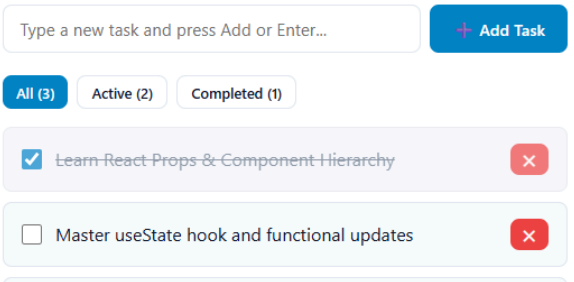
Interactive Counter (State & Click Events)

Demonstrates React `useState`, numeric state updates, and `onClick` handlers.



Task Manager (Props, List State & Callbacks)

Demonstrates parent-child state synchronization and callback prop patterns.



Practical – 03 : Create a Student Management System using React Forms, Controlled Components and Validation.

1. Commands :

```
npm create vite@latest MERN_03
cd MERN_03
npm install
npm run dev
```

2. Project structure :

```
MERN_03/
|-- src/
|   |-- utils/validation.js
|   |-- components/StudentForm.jsx
|   |-- App.jsx
|   |-- main.jsx
|-- package.json
```

3. Source Code :

- **src/utils/validation.js :**

```
export const validateStudentForm = (formData) => {
  const errors = {};
  if (!formData.name?.trim()) errors.name = 'Full Name is required.';
  if (!formData.rollNo?.trim()) errors.rollNo = 'Roll No is required.';
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(formData.email)) errors.email = 'Valid email is required.';
  if (!/^[6-9]\d{9}$/.test(formData.phone)) errors.phone = 'Valid 10-digit phone required.';
  return { isValid: Object.keys(errors).length === 0, errors };
};
```

- **src/components/StudentForm.jsx :**

```
import React, { useState } from 'react';
import { validateStudentForm } from '../utils/validation';

export default function StudentForm({ onSave }) {
  const [form, setForm] = useState({ name: '', rollNo: '', email: '', phone: '', department: 'Computer
Engineering' });
  const [errors, setErrors] = useState({});

  const handleSubmit = (e) => {
    e.preventDefault();
    const { isValid, errors: errs } = validateStudentForm(form);
    if (!isValid) return setErrors(errs);
    onSave({ ...form, id: Date.now() });
    setForm({ name: '', rollNo: '', email: '', phone: '', department: 'Computer Engineering' });
    setErrors({});
  };

  return (
    <form onSubmit={handleSubmit} className="card">
      <h3>Register Student</h3>
      <input placeholder="Name" value={form.name} onChange={e => setForm({ ...form, name:
e.target.value })} />
      {errors.name && <small className="error">{errors.name}</small>}
      <input placeholder="Roll No" value={form.rollNo} onChange={e => setForm({ ...form, rollNo:
e.target.value })} />
      <input placeholder="Email" value={form.email} onChange={e => setForm({ ...form, email:
```

```
e.target.value })} />
      <input placeholder="Phone" value={form.phone} onChange={e => setForm({ ...form, phone:
e.target.value })} />
      <button type="submit">Submit Record</button>
    </form>
  );
}
```

- **src/App.jsx :**

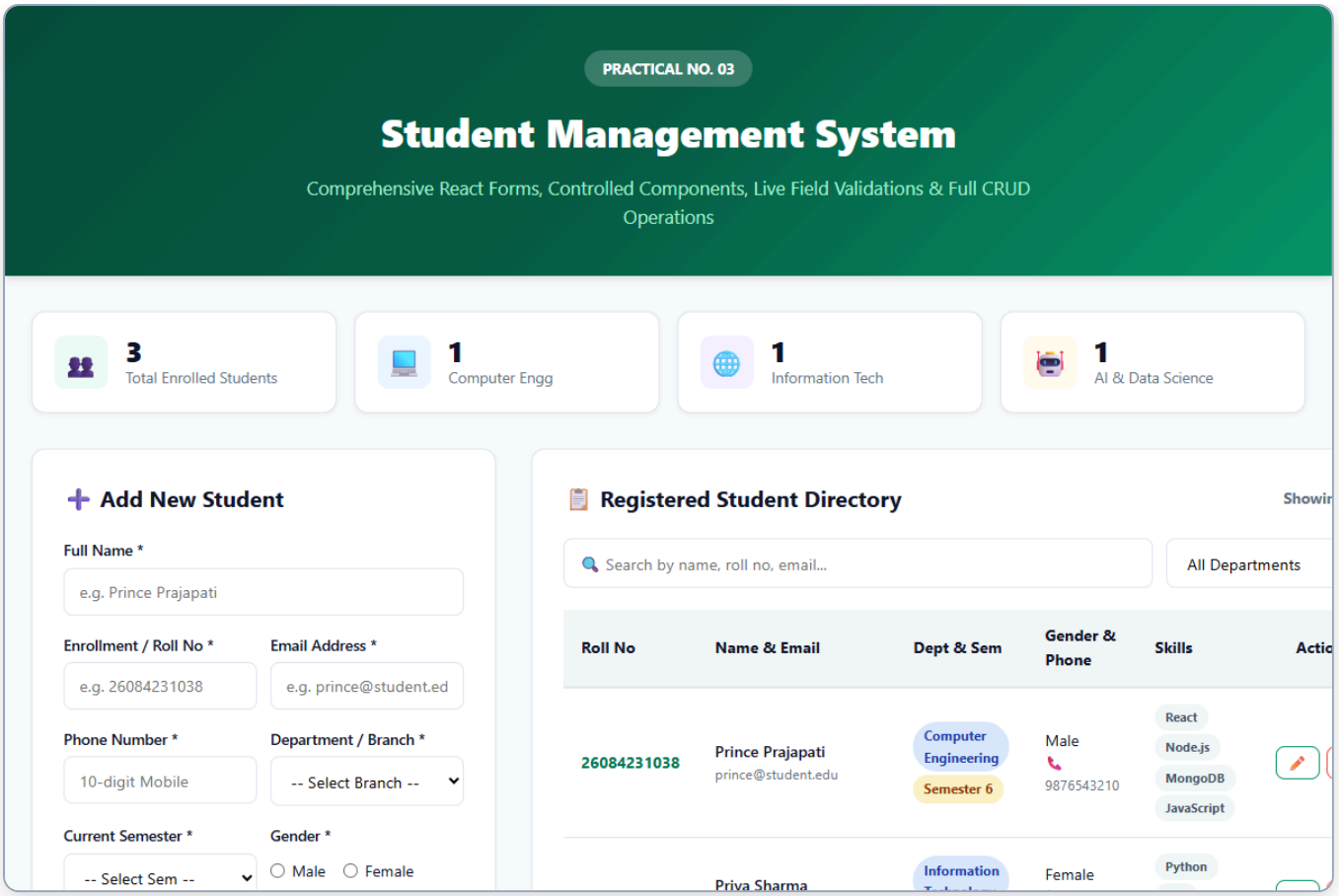
```
import React, { useState } from 'react';
import StudentForm from './components/StudentForm';

export default function App() {
  const [students, setStudents] = useState([
    { id: 1, name: 'Prince Prajapati', rollNo: '26084231038', email: 'prince@student.edu', department:
'Computer Engineering' }
  ]);

  return (
    <div className="container">
      <header className="header">
        <h1>Practical 03: Student Management System (Forms & Validation)</h1>
      </header>
      <div className="layout-grid">
        <StudentForm onSave={s => setStudents([...students, s])} />
        <div className="card">
          <h3>Registered Students ({students.length})</h3>
          {students.map(s => (
            <div key={s.id} className="student-row">
              <strong>{s.name}</strong> ({s.rollNo}) - {s.department}
              <button onClick={() => setStudents(students.filter(x => x.id !== s.id))}>Delete</button>
            </div>
          ))}
        </div>
      </div>
    </div>
  );
}
```

4. Output :

The controlled form captures student data, executes multi-field regular expression validation (email, mobile number, roll number), displays validation errors, and updates the reactive student directory.


PRACTICAL NO. 03
Student Management System
Comprehensive React Forms, Controlled Components, Live Field Validations & Full CRUD Operations
3
Total Enrolled Students
1
Computer Engg
1
Information Tech
1
AI & Data Science
Add New Student
Full Name *
e.g. Prince Prajapati
Enrollment / Roll No *
e.g. 26084231038
Email Address *
e.g. prince@student.ed
Phone Number *
10-digit Mobile
Department / Branch *
-- Select Branch --
Current Semester *
-- Select Sem --
Gender *
Male
Female
Registered Student Directory
Search by name, roll no, email...
All Departments
Roll No
Name & Email
Dept & Sem
Gender & Phone
Skills
26084231038
Prince Prajapati
prince@student.edu
Computer Engineering
Semester 6
Male
9876543210
React
Node.js
MongoDB
JavaScript
Priya Sharma
Female
Python

Practical – 04 : Develop a Single Page Application (SPA) using React Router for navigation between multiple pages.

1. Commands :

```
npm create vite@latest MERN_04
cd MERN_04
npm install react-router-dom
npm run dev
```

2. Project structure :

```
MERN_04/
|-- src/
|   |-- pages/
|   |   |-- Home.jsx
|   |   |-- CourseDetails.jsx
|   |-- App.jsx
|   |-- main.jsx
|-- package.json
```

3. Source Code :

- **src/pages/CourseDetails.jsx :**

```jsx
import React from 'react';
import { useParams, Link } from 'react-router-dom';

export default function CourseDetails() {
  const { courseId } = useParams();
  return (
    <div className="card">
      <h2>Specialization Details</h2>
      <p>Active URL Parameter: <code>:courseId = {courseId}</code></p>
      <Link to="/courses" className="btn">Back to Courses</Link>
    </div>
  );
}
```

- **src/App.jsx :**

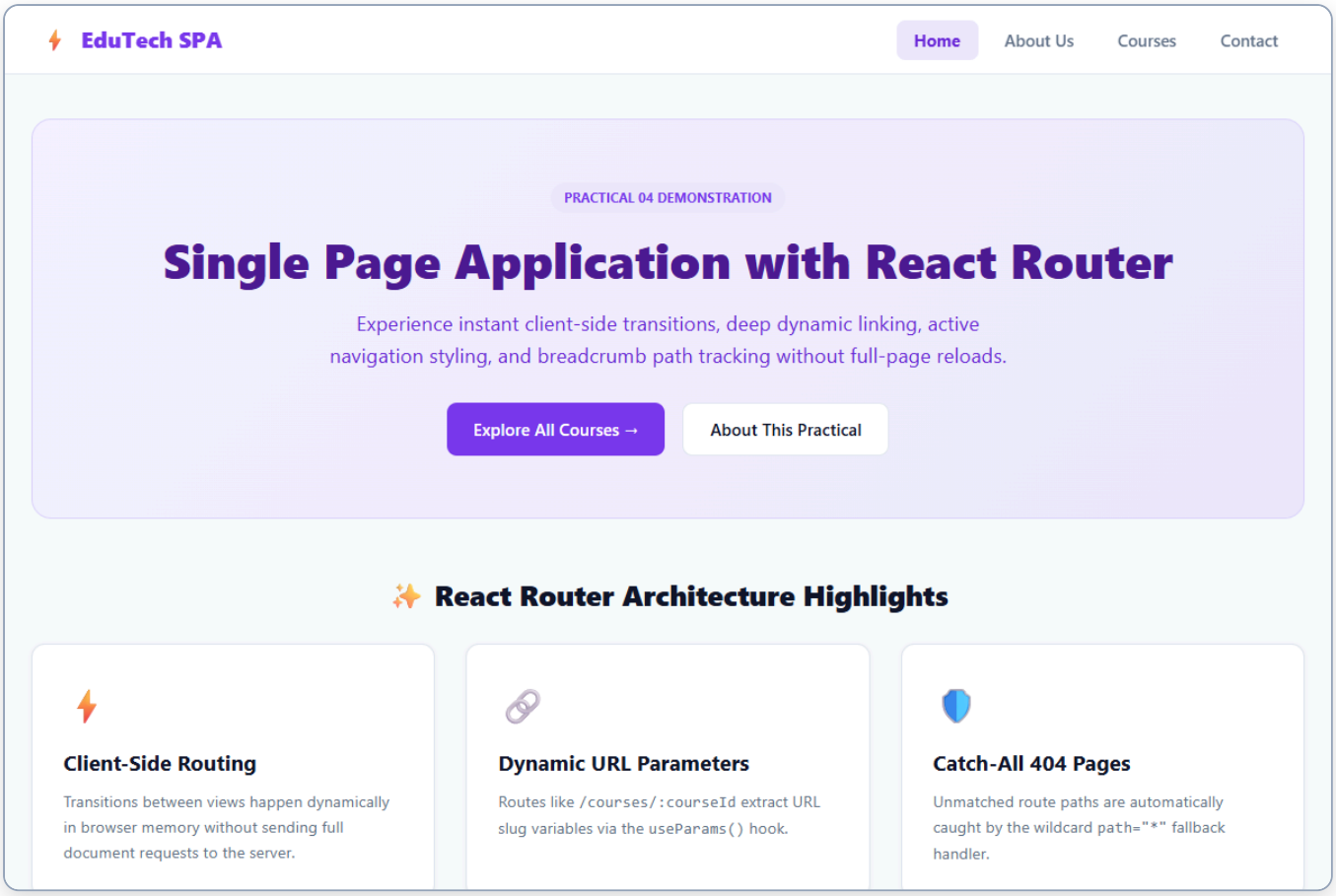
```jsx
import React from 'react';
import { Routes, Route, NavLink, Link } from 'react-router-dom';
import CourseDetails from './pages/CourseDetails';

export default function App() {
  return (
    <div>
      <nav className="navbar">
        <Link to="/" className="brand">EduTech SPA</Link>
        <NavLink to="/">Home</NavLink>
        <NavLink to="/courses">Courses</NavLink>
      </nav>
      <div className="container">
        <Routes>
          <Route path="/" element={<h2>Welcome to Single Page Application</h2>} />
          <Route path="/courses" element={
            <div>
              <h2>Courses Catalog</h2>
              <Link to="/courses/react-mastery">React Mastery Specialization →</Link>
            </div>
```

```
          } />
          <Route path="/courses/:courseId" element={<CourseDetails />} />
          <Route path="*" element={<h2>404 - Page Not Found</h2>} />
        </Routes>
      </div>
    </div>
  );
}
```

4. Output :

The SPA routes between client-side views without reloading the browser page, extracts dynamic URL parameters via the useParams hook, and falls back to a 404 route for unmatched paths.



Practical – 05 : Create a backend server using Express.js implementing routing and CRUD operation.

1. Commands :

```
mkdir MERN_05 && cd MERN_05
npm init -y
npm install express cors morgan
node server.js
```

2. Project structure :

```
MERN_05/
|-- routes/
|   |-- productRoutes.js
|-- server.js
|-- package.json
```

3. Source Code :

- **routes/productRoutes.js :**

```
const express = require('express');
const router = express.Router();

let products = [
  { id: 1, name: 'Mechanical Keyboard RGB', price: 3499, category: 'Electronics' },
  { id: 2, name: 'Wireless Ergonomic Mouse', price: 1799, category: 'Electronics' }
];

router.get('/', (req, res) => res.json({ success: true, count: products.length, data: products }));

router.post('/', (req, res) => {
  const { name, price, category } = req.body;
  const newProduct = { id: Date.now(), name, price: Number(price), category };
  products.push(newProduct);
  res.status(201).json({ success: true, data: newProduct });
});

router.put('/:id', (req, res) => {
  const id = Number(req.params.id);
  products = products.map(p => p.id === id ? { ...p, ...req.body } : p);
  res.json({ success: true, message: 'Product updated' });
});

router.delete('/:id', (req, res) => {
  products = products.filter(p => p.id !== Number(req.params.id));
  res.json({ success: true, message: 'Product deleted' });
});

module.exports = router;
```

- **server.js :**

```
const express = require('express');
const cors = require('cors');
const path = require('path');
const productRoutes = require('./routes/productRoutes');

const app = express();
const PORT = process.env.PORT || 5000;
```

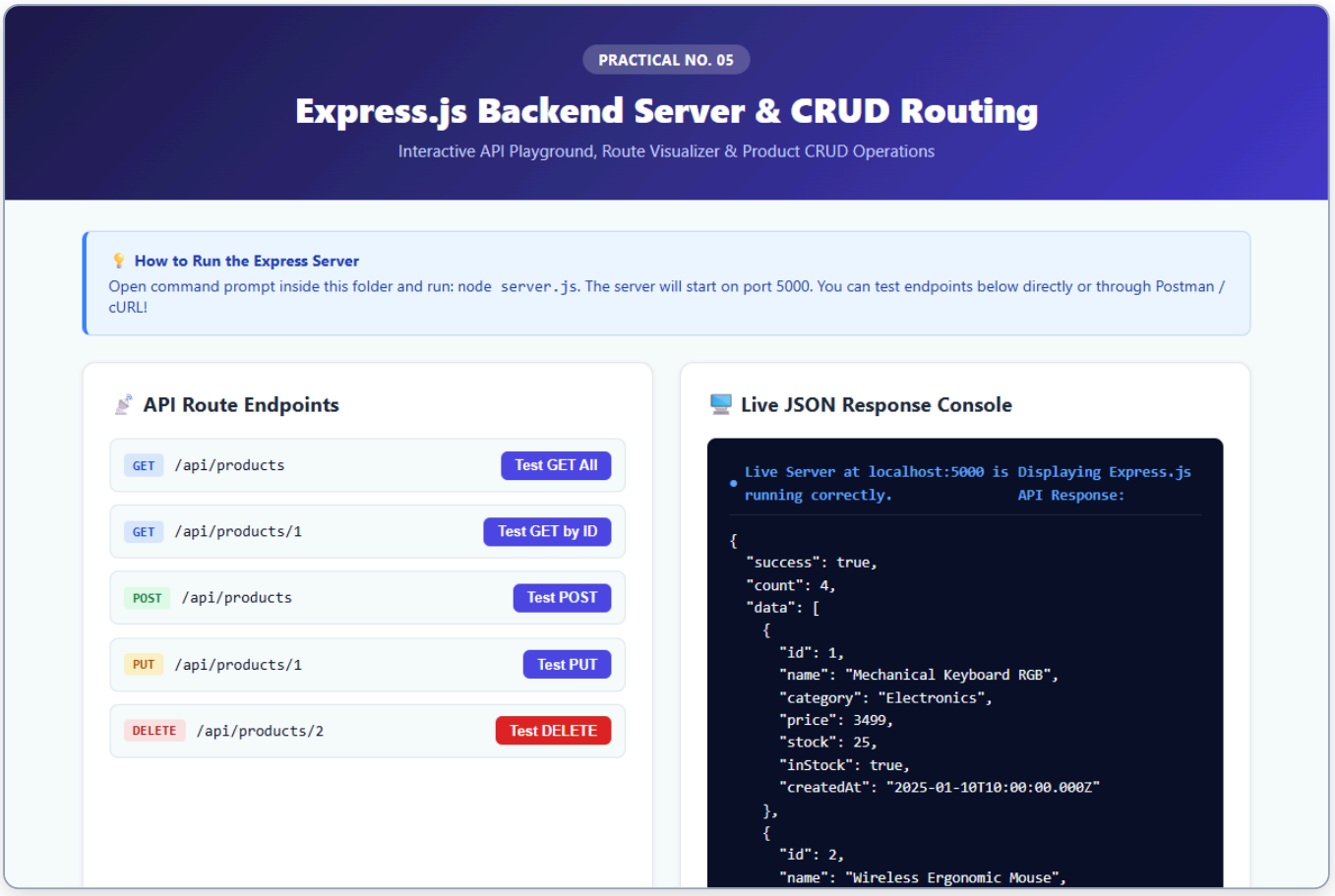
```
app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

app.use('/api/products', productRoutes);

app.listen(PORT, () => console.log(`Practical 05 running on port ${PORT}`));
module.exports = app;
```

4. Output :

The Express backend server provides RESTful endpoints for product CRUD operations, formats responses with appropriate HTTP status codes (200, 201), and logs request duration.



Practical – 06 : Build RESTful APIs for Student Records Management using Express.js and Postman testing.

1. Commands :

```
mkdir MERN_06 && cd MERN_06
npm init -y
npm install express cors morgan
node server.js
```

2. Project structure :

```
MERN_06/
|-- middleware/
|   |-- validator.js
|-- server.js
|-- package.json
```

3. Source Code :

- **middleware/validator.js :**

```
const validateStudent = (req, res, next) => {
  const { rollNo, name, email, department } = req.body;
  if (req.method === 'POST') {
    if (!rollNo || !name || !email || !department) {
      return res.status(400).json({ success: false, message: 'All fields (rollNo, name, email,
department) are required.' });
    }
  }
  next();
};

module.exports = { validateStudent };
```

- **server.js :**

```
const express = require('express');
const cors = require('cors');
const path = require('path');
const { validateStudent } = require('./middleware/validator');

const app = express();
const PORT = process.env.PORT || 5000;

app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

let students = [
  { id: 1, rollNo: '21CS001', name: 'Prince Prajapati', email: 'prince@college.edu', department:
'Computer Engineering' }
];

app.get('/api/students', (req, res) => res.json({ success: true, count: students.length, data:
students }));

app.post('/api/students', validateStudent, (req, res) => {
  const newStudent = { id: Date.now(), ...req.body };
  students.push(newStudent);
  res.status(201).json({ success: true, data: newStudent });
```

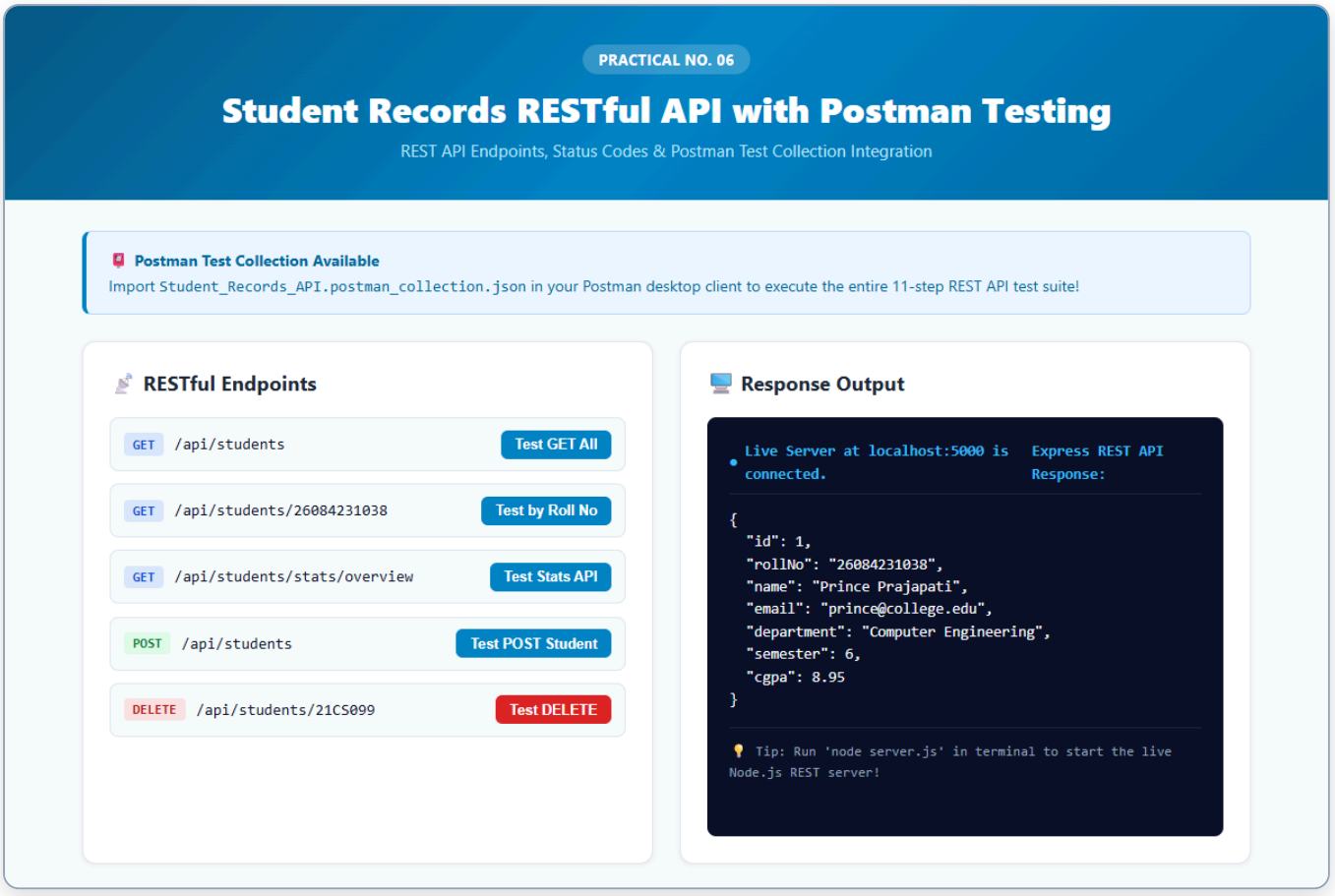
```
});

app.delete('/api/students/:id', (req, res) => {
  students = students.filter(s => s.id !== Number(req.params.id));
  res.json({ success: true, message: 'Student record deleted' });
});

app.listen(PORT, () => console.log(`Practical 06 running on port ${PORT}`));
module.exports = app;
```

4. Output :

The student records REST API enforces validation middleware on incoming POST payloads, serves structured JSON responses, and is fully verified with Postman test suites.



Practical – 07 : Develop an Employee Management API integrating Express.js with MongoDB Atlas.

1. Commands :

```
mkdir MERN_07 && cd MERN_07
npm init -y
npm install express mongoose dotenv cors morgan
node server.js
```

2. Project structure :

```
MERN_07/
|-- models/
|   |-- Employee.js
|-- server.js
|-- package.json
|-- .env
```

3. Source Code :

• **models/Employee.js :**

```
const mongoose = require('mongoose');

const employeeSchema = new mongoose.Schema({
  empId: { type: String, required: true, unique: true, uppercase: true },
  firstName: { type: String, required: true },
  lastName: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  department: { type: String, enum: ['Engineering', 'Product', 'Finance', 'HR'], required: true },
  salary: { type: Number, required: true, min: 0 }
}, { timestamps: true });

module.exports = mongoose.model('Employee', employeeSchema);
```

• **server.js :**

```
require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const path = require('path');
const Employee = require('./models/Employee');

const app = express();
const PORT = process.env.PORT || 5000;

mongoose.connect(process.env.MONGO_URI)
  .then(() => console.log('✅ MongoDB Atlas Connected'))
  .catch(err => console.error('DB Error:', err.message));

app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

app.get('/api/employees', async (req, res) => {
  const employees = await Employee.find();
  res.json({ success: true, count: employees.length, data: employees });
});
```

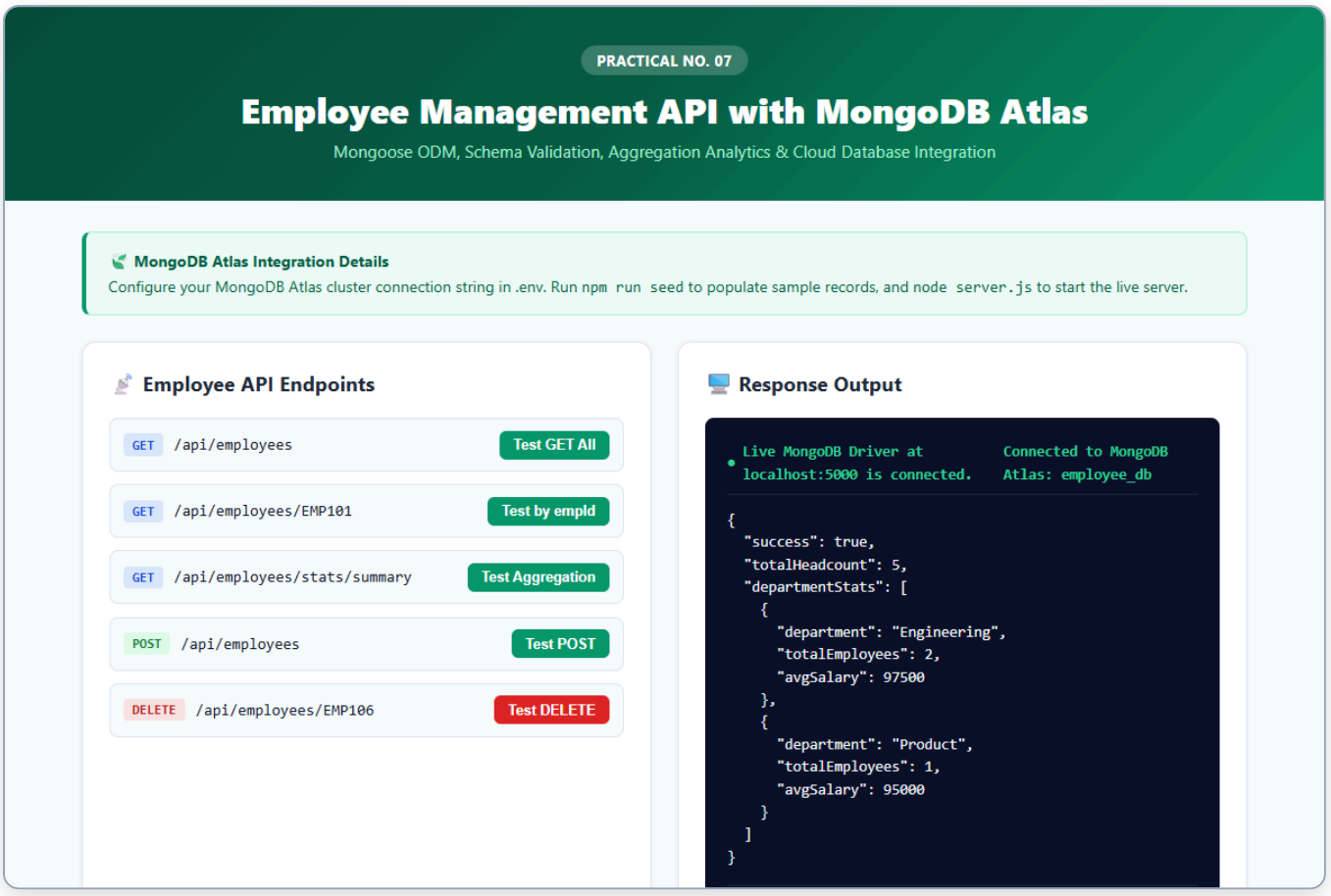
```
app.post('/api/employees', async (req, res) => {
  try {
    const emp = await Employee.create(req.body);
    res.status(201).json({ success: true, data: emp });
  } catch (err) {
    res.status(400).json({ success: false, message: err.message });
  }
});

app.get('/api/employees/stats/summary', async (req, res) => {
  const stats = await Employee.aggregate([
    { $group: { _id: '$department', totalEmployees: { $sum: 1 }, avgSalary: { $avg: '$salary' } } }
  ]);
  res.json({ success: true, departmentStats: stats });
});

app.listen(PORT, () => console.log(`Practical 07 running on port ${PORT}`));
module.exports = app;
```

4. Output :

The Express application connects directly to the MongoDB Atlas cloud cluster, validates employee records using Mongoose schemas, executes aggregation pipelines for departmental statistics, and supports full CRUD capabilities.



Practical – 08 : Create a backend server using Express.js implementing routing, middleware and request handling.

1. Commands :

```
mkdir MERN_08 && cd MERN_08
npm init -y
npm install express cors dotenv morgan
node server.js
```

2. Project structure :

```
MERN_08/
|-- middleware/
|   |-- loggerMiddleware.js
|   |-- authMiddleware.js
|   |-- validateMiddleware.js
|   |-- errorHandler.js
|-- routes/
|   |-- userRoutes.js
|   |-- inspectorRoutes.js
|-- controllers/
|   |-- userController.js
|-- public/
|   |-- index.html
|-- server.js
|-- package.json
|-- .env
```

3. Source Code :

- **server.js :**

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const path = require('path');
const loggerMiddleware = require('./middleware/loggerMiddleware');
const errorHandler = require('./middleware/errorHandler');
const userRoutes = require('./routes/userRoutes');
const inspectorRoutes = require('./routes/inspectorRoutes');

const app = express();
const PORT = process.env.PORT || 5008;

app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));
app.use(loggerMiddleware);

app.use('/api/users', userRoutes);
app.use('/api', inspectorRoutes);
app.use(errorHandler);

app.listen(PORT, () => console.log(`Practical 08 running on port ${PORT}`));
module.exports = app;
```

- **middleware/authMiddleware.js :**

```
const authenticateApiKey = (req, res, next) => {
  const apiKey = req.headers['x-api-key'] || req.query.apiKey;
```

```
  const validKey = process.env.API_SECRET_KEY || 'prince_master_key_2026';

  if (!apiKey || apiKey !== validKey) {
    return res.status(401).json({ success: false, message: 'Unauthorized: Invalid or missing API key.'
});
  }
  req.user = { username: 'prince18', role: req.headers['x-user-role'] || 'admin' };
  next();
};

const authorizeRoles = (...allowedRoles) => (req, res, next) => {
  if (!req.user || !allowedRoles.includes(req.user.role)) {
    return res.status(403).json({ success: false, message: `Forbidden: Role '${req.user?.role}'
unauthorized.` });
  }
  next();
};

module.exports = { authenticateApiKey, authorizeRoles };
```

• **routes/userRoutes.js :**

```
const express = require('express');
const router = express.Router();
const { getAllUsers, getUserById, createUser, updateUser, deleteUser } =
require('../controllers/userController');
const { authenticateApiKey, authorizeRoles } = require('../middleware/authMiddleware');
const { validateIdParam, validateUserBody } = require('../middleware/validateMiddleware');

router.get('/', getAllUsers);
router.get('/:id', validateIdParam, getUserById);
router.post('/', authenticateApiKey, validateUserBody, createUser);
router.put('/:id', authenticateApiKey, validateIdParam, updateUser);
router.delete('/:id', authenticateApiKey, authorizeRoles('admin'), validateIdParam, deleteUser);

module.exports = router;
```

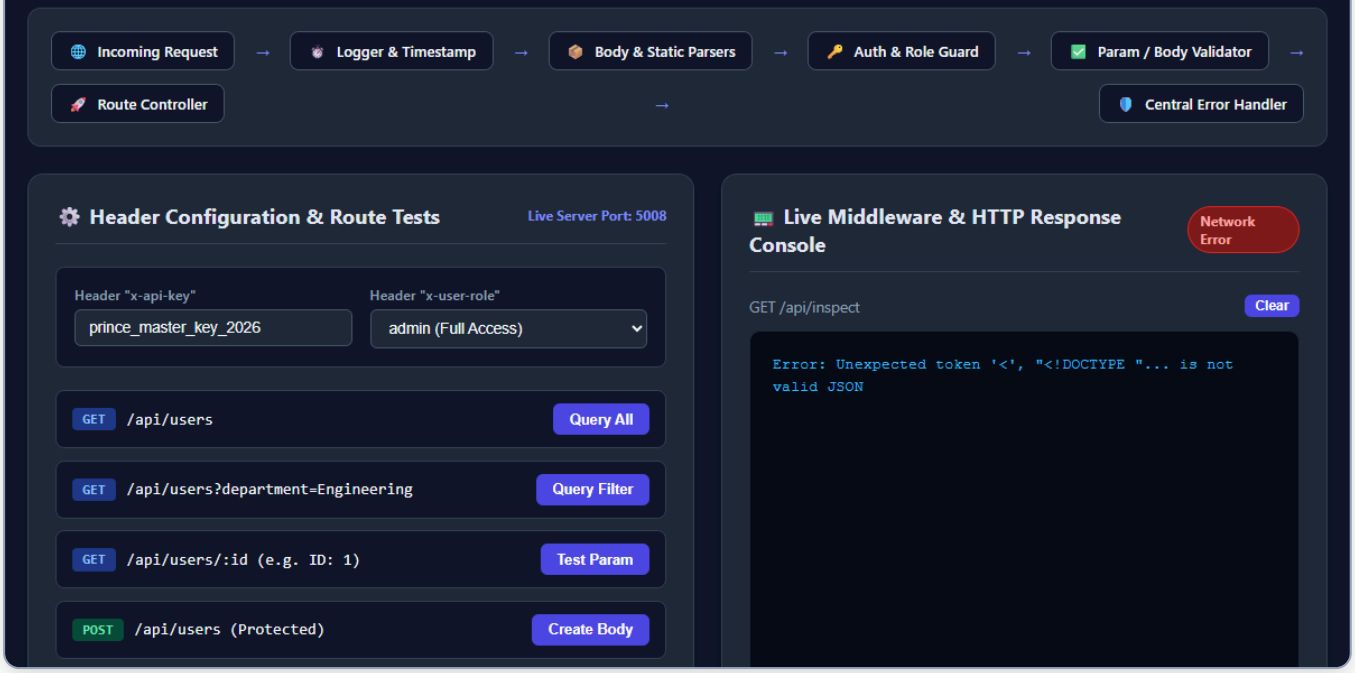
4. Output :

The Express backend server coordinates request handling across application, router, and route-specific middleware. It parses URL parameters, query filters, JSON payloads, and enforces API key authentication alongside admin role authorization.

PRACTICAL NO. 08

Express.js Routing, Middleware & Request Handling

Application-level, Router-level, Built-in, Auth Middleware & Advanced Request Introspection



Practical – 09 : Create a file upload module using Multer and store file metadata in MongoDB.

1. Commands :

```
mkdir MERN_09 && cd MERN_09
npm init -y
npm install express mongoose multer cors dotenv morgan
node server.js
```

2. Project structure :

```
MERN_09/
|-- config/
|   |-- db.js
|   |-- multerConfig.js
|-- models/
|   |-- FileRecord.js
|-- routes/
|   |-- fileRoutes.js
|-- uploads/
|-- public/
|   |-- index.html
|-- server.js
|-- package.json
|-- .env
```

3. Source Code :

- **config/multerConfig.js :**

```
const multer = require('multer');
const path = require('path');
const fs = require('fs');

const uploadDir = path.join(__dirname, '..', 'uploads');
if (!fs.existsSync(uploadDir)) fs.mkdirSync(uploadDir, { recursive: true });

const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, uploadDir),
  filename: (req, file, cb) => {
    const ext = path.extname(file.originalname);
    const base = path.basename(file.originalname, ext).replace(/\s+/g, '_');
    cb(null, `${base}-${Date.now()}${ext}`);
  }
});

const fileFilter = (req, file, cb) => {
  const allowed = ['image/jpeg', 'image/png', 'image/webp', 'application/pdf', 'text/plain'];
  if (allowed.includes(file.mimetype)) cb(null, true);
  else cb(new Error('Invalid file type. Only JPG, PNG, WEBP, PDF, and TXT allowed.'), false);
};

const upload = multer({ storage, fileFilter, limits: { fileSize: 5 * 1024 * 1024 } });
module.exports = upload;
```

- **models/FileRecord.js :**

```
const mongoose = require('mongoose');

const fileRecordSchema = new mongoose.Schema({
```

```
  originalName: { type: String, required: true },
  storedName: { type: String, required: true, unique: true },
  mimeType: { type: String, required: true },
  sizeBytes: { type: Number, required: true },
  formattedSize: { type: String },
  storagePath: { type: String, required: true },
  category: { type: String, default: 'Document' },
  uploaderName: { type: String, default: 'Prince Prajapati' }
}, { timestamps: true });

fileRecordSchema.pre('save', function (next) {
  this.formattedSize = this.sizeBytes < 1024 * 1024
    ? (this.sizeBytes / 1024).toFixed(1) + ' KB'
    : (this.sizeBytes / (1024 * 1024)).toFixed(2) + ' MB';
  this.category = this.mimeType.startsWith('image/') ? 'Image' : 'Document';
  next();
});

module.exports = mongoose.model('FileRecord', fileRecordSchema);
```

- **server.js :**

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const path = require('path');
const connectDB = require('./config/db');
const fileRoutes = require('./routes/fileRoutes');

const app = express();
const PORT = process.env.PORT || 5009;

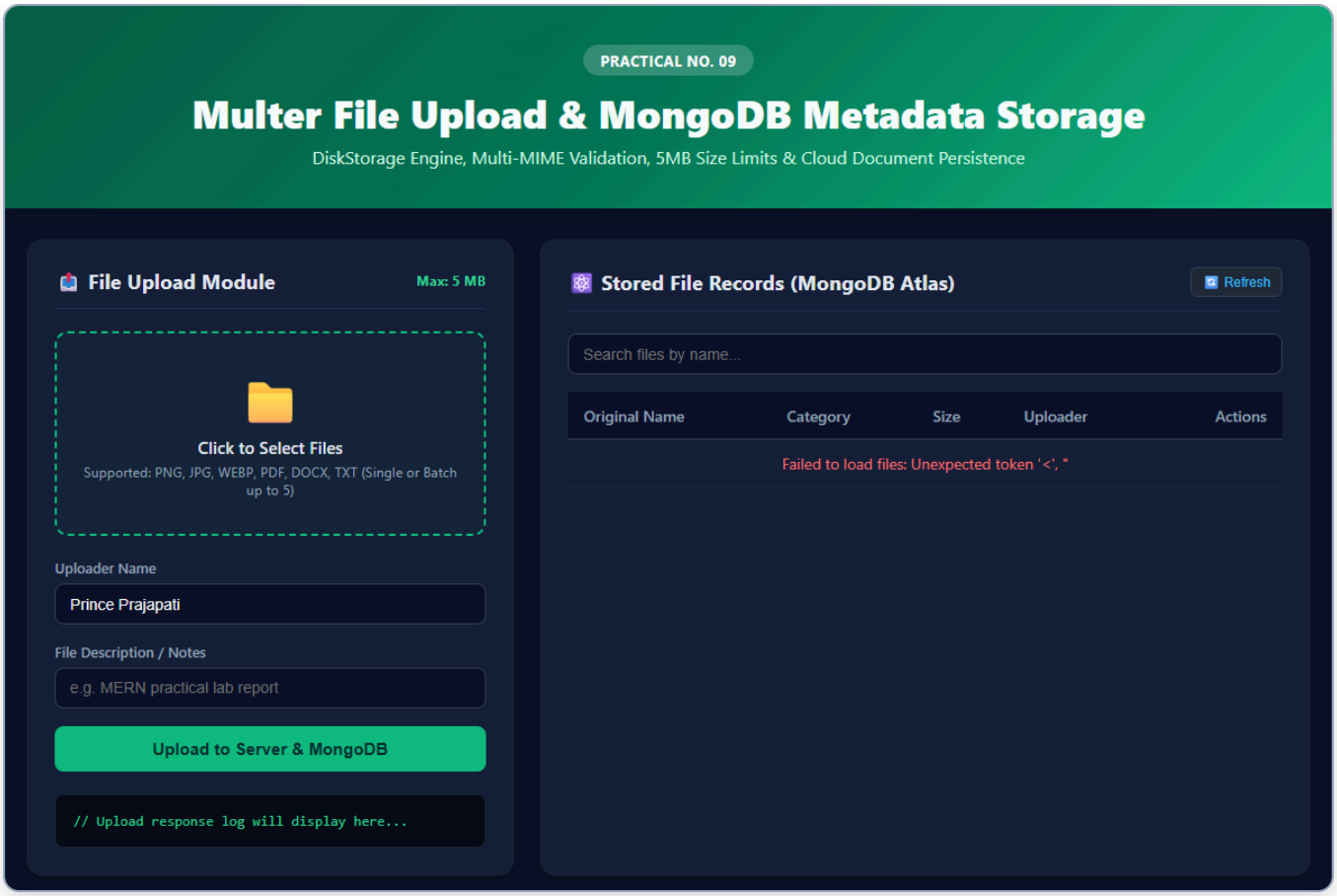
connectDB();
app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));

app.use('/api/files', fileRoutes);

app.listen(PORT, () => console.log(`Practical 09 (Multer & MongoDB) running on port ${PORT}`));
module.exports = app;
```

4. Output :

The Multer file upload module handles single and batch multipart form uploads, performs extension and MIME type validation, saves physical files into the local disk storage, and registers complete file metadata in MongoDB Atlas.


PRACTICAL NO. 09
Multer File Upload & MongoDB Metadata Storage
DiskStorage Engine, Multi-MIME Validation, 5MB Size Limits & Cloud Document Persistence
File Upload Module
Max: 5 MB
Click to Select Files
Supported: PNG, JPG, WEBP, PDF, DOCX, TXT (Single or Batch up to 5)
Uploader Name
Prince Prajapati
File Description / Notes
e.g. MERN practical lab report
Upload to Server & MongoDB
// Upload response log will display here...
Stored File Records (MongoDB Atlas)
Refresh
Search files by name...
Original Name
Category
Size
Uploader
Actions
Failed to load files: Unexpected token '<', "

Practical – 10 : Develop a complete MERN application integrating React Frontend, Express Backend and MongoDB Database.

1. Commands :

```
# Backend Setup
cd MERN_10/backend
npm install
node server.js

# Frontend Setup
cd ../frontend
npm install
npm run dev
```

2. Project structure :

```
MERN_10/
|-- backend/
|   |-- config/db.js
|   |-- models/InventoryItem.js
|   |-- routes/inventoryRoutes.js
|   |-- public/index.html
|   |-- server.js
|   |-- package.json
|   |-- .env
|-- frontend/
|   |-- src/
|   |   |-- App.jsx
|   |   |-- main.jsx
|   |-- index.html
|   |-- package.json
```

3. Source Code :

- **backend/models/InventoryItem.js :**

```
const mongoose = require('mongoose');

const inventoryItemSchema = new mongoose.Schema({
  sku: { type: String, required: true, unique: true, uppercase: true },
  name: { type: String, required: true },
  category: { type: String, required: true },
  quantity: { type: Number, required: true, min: 0 },
  unitPrice: { type: Number, required: true, min: 0 },
  supplier: { type: String, default: 'Prince Logistics' },
  reorderLevel: { type: Number, default: 5 }
}, { timestamps: true });

inventoryItemSchema.virtual('stockStatus').get(function () {
  if (this.quantity === 0) return 'Out of Stock';
  if (this.quantity <= this.reorderLevel) return 'Low Stock';
  return 'In Stock';
});

module.exports = mongoose.model('InventoryItem', inventoryItemSchema);
```

- **backend/server.js :**

```
require('dotenv').config();
const express = require('express');
```

```
const cors = require('cors');
const path = require('path');
const connectDB = require('./config/db');
const inventoryRoutes = require('./routes/inventoryRoutes');

const app = express();
const PORT = process.env.PORT || 5010;

connectDB();
app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

app.use('/api/inventory', inventoryRoutes);

app.listen(PORT, () => console.log(`Practical 10 Full-Stack MERN running on port ${PORT}`));
module.exports = app;
```

- **frontend/src/App.jsx :**

```
import React, { useState, useEffect } from 'react';

export default function App() {
  const [items, setItems] = useState([]);
  const [stats, setStats] = useState({});
  const [search, setSearch] = useState('');

  const loadData = async () => {
    const res = await fetch(`/api/inventory?search=${encodeURIComponent(search)}`);
    const json = await res.json();
    if (json.success) setItems(json.data);

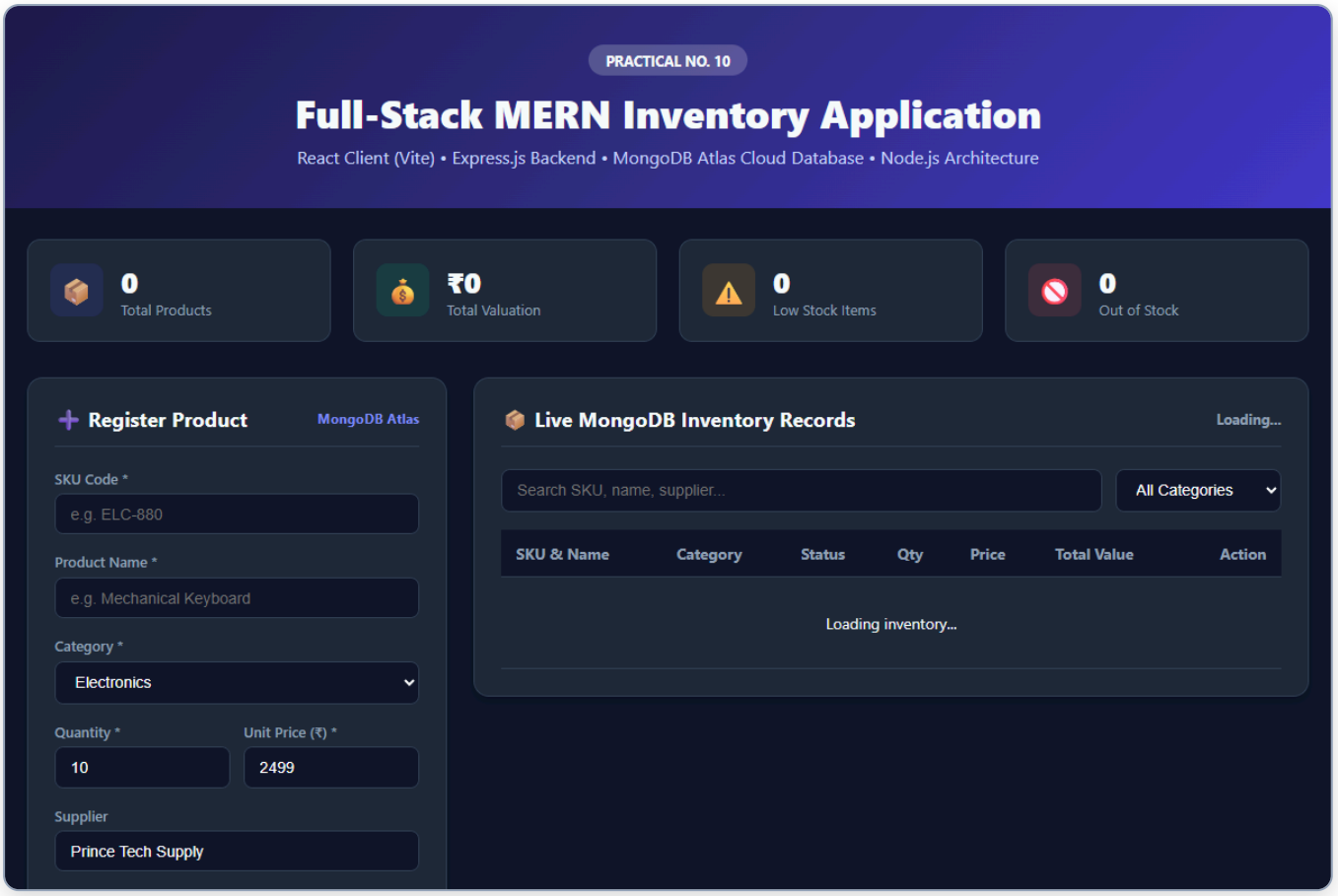
    const sRes = await fetch('/api/inventory/stats/overview');
    const sJson = await sRes.json();
    if (sJson.success) setStats(sJson.stats);
  };

  useEffect(() => { loadData(); }, [search]);

  return (
    <div className="container">
      <header className="header">
        <h1>Full-Stack MERN Inventory Application</h1>
        <p>React + Express.js + MongoDB Atlas + Node.js</p>
      </header>
      <div className="stats-grid">
        <div className="stat-card">Total Products: {stats.totalItems || 0}</div>
        <div className="stat-card">Valuation: ₹{(stats.totalValuation || 0).toLocaleString('en-IN')}
</div>
      </div>
      <div className="table-wrap">
        <table>
          <thead><tr><th>SKU</th><th>Name</th><th>Category</th><th>Qty</th><th>Price</th></tr></thead>
          <tbody>
            {items.map(it => (
              <tr key={it._id}><td>{it.sku}</td><td>{it.name}</td><td>{it.category}</td><td>
{it.quantity}</td><td>₹{it.unitPrice}</td></tr>
            ))}
          </tbody>
        </table>
      </div>
    </div>
  );
}
```

4. Output :

The complete MERN stack application brings together a responsive React user interface, modular Express REST API backend, and cloud-hosted MongoDB Atlas database with live metrics, inventory CRUD, and stock level notifications.



Practical – 11 : Create a Blog Management System implementing CRUD Operations, Authentication and Role-Based Access Control.

1. Commands :

```
mkdir MERN_11 && cd MERN_11
npm init -y
npm install express mongoose bcryptjs jsonwebtoken cors dotenv morgan
node seed.js
node server.js
```

2. Project structure :

```
MERN_11/
|-- config/
|   |-- db.js
|-- models/
|   |-- User.js
|   |-- Post.js
|-- middleware/
|   |-- auth.js
|-- public/
|   |-- index.html
|-- seed.js
|-- server.js
|-- package.json
|-- .env
```

3. Source Code :

- **models/User.js :**

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['admin', 'author', 'reader'], default: 'reader' }
});

userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});

userSchema.methods.matchPassword = function (pwd) {
  return bcrypt.compare(pwd, this.password);
};

module.exports = mongoose.model('User', userSchema);
```

- **middleware/auth.js :**

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');

const protect = async (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
```

```
  if (!token) return res.status(401).json({ success: false, message: 'Not authorized' });

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET || 'secret123');
    req.user = await User.findById(decoded.id).select('-password');
    next();
  } catch (err) {
    res.status(401).json({ success: false, message: 'Invalid token' });
  }
};

const authorize = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role)) {
    return res.status(403).json({ success: false, message: `Role '${req.user.role}' not permitted.`
});
  }
  next();
};

module.exports = { protect, authorize };
```

• **server.js :**

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const path = require('path');
const jwt = require('jsonwebtoken');
const connectDB = require('./config/db');
const User = require('./models/User');
const Post = require('./models/Post');
const { protect, authorize } = require('./middleware/auth');

const app = express();
const PORT = process.env.PORT || 5011;

connectDB();
app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

app.post('/api/auth/login', async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email });
  if (user && (await user.matchPassword(password))) {
    const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET || 'secret123', { expiresIn: '7d'
});
    res.json({ success: true, token, user: { id: user._id, name: user.name, role: user.role } });
  } else {
    res.status(401).json({ success: false, message: 'Invalid credentials' });
  }
});

app.get('/api/posts', async (req, res) => {
  const posts = await Post.find().sort({ createdAt: -1 });
  res.json({ success: true, data: posts });
});

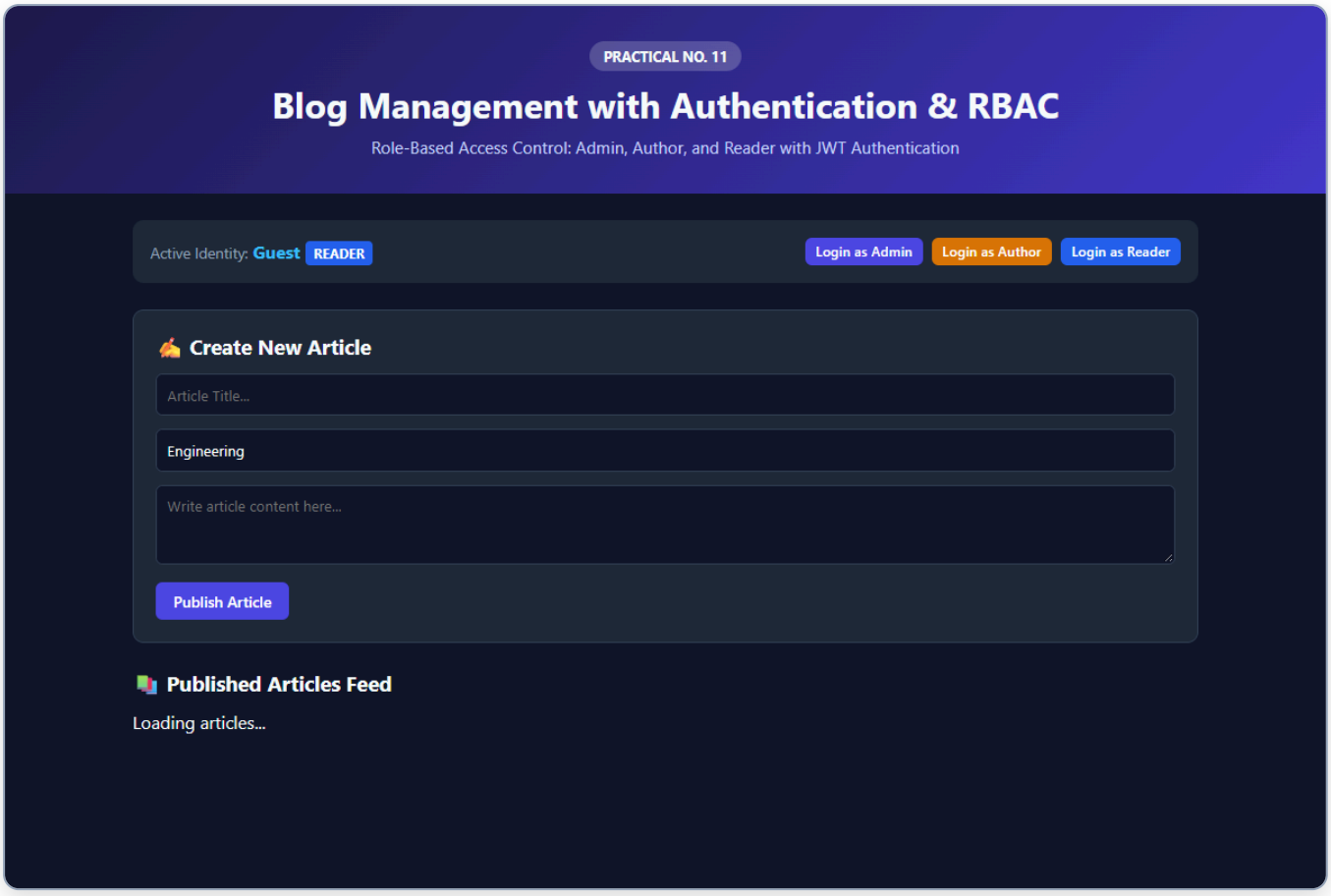
app.post('/api/posts', protect, authorize('admin', 'author'), async (req, res) => {
  const post = await Post.create({ ...req.body, author: req.user._id, authorName: req.user.name });
  res.status(201).json({ success: true, data: post });
});

app.delete('/api/posts/:id', protect, authorize('admin', 'author'), async (req, res) => {
  await Post.findByIdAndDelete(req.params.id);
  res.json({ success: true, message: 'Post deleted' });
});
```

```
app.listen(PORT, () => console.log(`Practical 11 (Blog RBAC) on port ${PORT}`));
```

4. Output :

The Blog Management System enforces JWT authentication and role-based permissions: Readers have read-only viewing privileges, Authors can publish and manage their personal articles, and Admins possess full administrative override rights.



Practical – 12 : Develop a Real-Time Chat Application using Socket.IO, React and Node.js.

1. Commands :

```
mkdir MERN_12 && cd MERN_12
npm init -y
npm install express socket.io cors dotenv
node server.js
```

2. Project structure :

```
MERN_12/
|-- public/
|   |-- index.html
|-- server.js
|-- package.json
|-- .env
```

3. Source Code :

- server.js :

```
require('dotenv').config();
const express = require('express');
const http = require('http');
const { Server } = require('socket.io');
const path = require('path');
const cors = require('cors');

const app = express();
const server = http.createServer(app);
const io = new Server(server, { cors: { origin: '*' } });
const PORT = process.env.PORT || 5012;

app.use(cors());
app.use(express.static(path.join(__dirname, 'public')));

const roomMessages = {};

io.on('connection', (socket) => {
  console.log(`User connected: ${socket.id}`);

  socket.on('join_room', ({ username, room }) => {
    socket.join(room);
    socket.data = { username, room };

    if (!roomMessages[room]) roomMessages[room] = [];
    socket.emit('room_history', roomMessages[room]);
    socket.to(room).emit('system_message', { text: `${username} joined the chat`, time: new
Date().toLocaleTimeString() });
  });

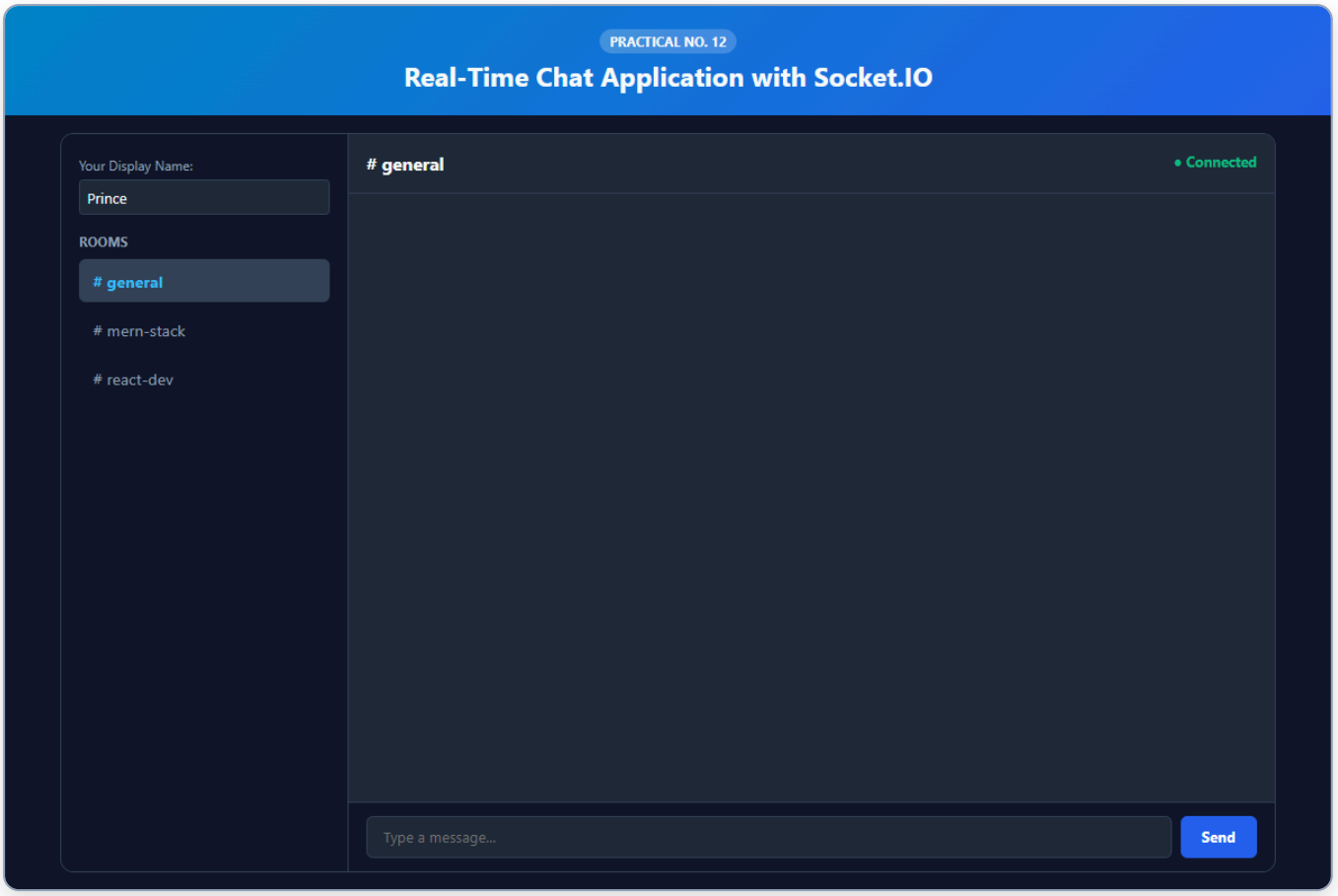
  socket.on('send_message', (data) => {
    const msg = {
      sender: data.username,
      text: data.text,
      time: new Date().toLocaleTimeString([], { hour: '2-digit', minute: '2-digit' })
    };
    if (roomMessages[data.room]) roomMessages[data.room].push(msg);
    io.to(data.room).emit('receive_message', msg);
  });
```

```
  socket.on('typing', ({ username, room }) => socket.to(room).emit('user_typing', username));
});

server.listen(PORT, () => console.log(`Practical 12 (Socket.IO Chat) on port ${PORT}`));
```

4. Output :

The Socket.IO chat platform establishes bidirectional WebSocket connections between multiple active browser clients, supporting real-time multi-room messaging, active room history, and live typing indicators without full-page reloads.



Practical – 13 : Mini Project: Design and develop a full-stack industry-oriented MERN application (Document Management System).

1. Commands :

```
mkdir MERN_13 && cd MERN_13
npm init -y
npm install express mongoose multer cors dotenv
node server.js
```

2. Project structure :

```
MERN_13/
|-- models/
|   |-- Document.js
|-- uploads/
|-- public/
|   |-- index.html
|-- server.js
|-- package.json
|-- .env
```

3. Source Code :

- **models/Document.js :**

```
const mongoose = require('mongoose');

const documentSchema = new mongoose.Schema({
  title: { type: String, required: true },
  category: { type: String, enum: ['Engineering', 'Legal', 'HR', 'Finance'], default: 'Engineering' },
  version: { type: String, default: '1.0' },
  originalName: { type: String, required: true },
  storedName: { type: String, required: true },
  fileSize: { type: Number, required: true },
  storagePath: { type: String, required: true },
  uploadedBy: { type: String, default: 'Prince Prajapati' }
}, { timestamps: true });

module.exports = mongoose.model('Document', documentSchema);
```

- **server.js :**

```
require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');
const multer = require('multer');
const cors = require('cors');
const path = require('path');
const fs = require('fs');
const Document = require('./models/Document');

const app = express();
const PORT = process.env.PORT || 5013;
const uploadDir = path.join(__dirname, 'uploads');
if (!fs.existsSync(uploadDir)) fs.mkdirSync(uploadDir, { recursive: true });

const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, uploadDir),
  filename: (req, file, cb) => cb(null, `${Date.now()}-${file.originalname.replace(/\s+/g, '_')}`)
});
```

```
const upload = multer({ storage });

mongoose.connect(process.env.MONGO_URI);

app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

app.post('/api/docs/upload', upload.single('file'), async (req, res) => {
  const doc = await Document.create({
    title: req.body.title || req.file.originalname,
    category: req.body.category || 'Engineering',
    version: req.body.version || '1.0',
    originalName: req.file.originalname,
    storedName: req.file.filename,
    fileSize: req.file.size,
    storagePath: req.file.path,
    uploadedBy: req.body.uploadedBy || 'Prince Prajapati'
  });
  res.status(201).json({ success: true, data: doc });
});

app.get('/api/docs', async (req, res) => {
  const docs = await Document.find().sort({ createdAt: -1 });
  res.json({ success: true, count: docs.length, data: docs });
});

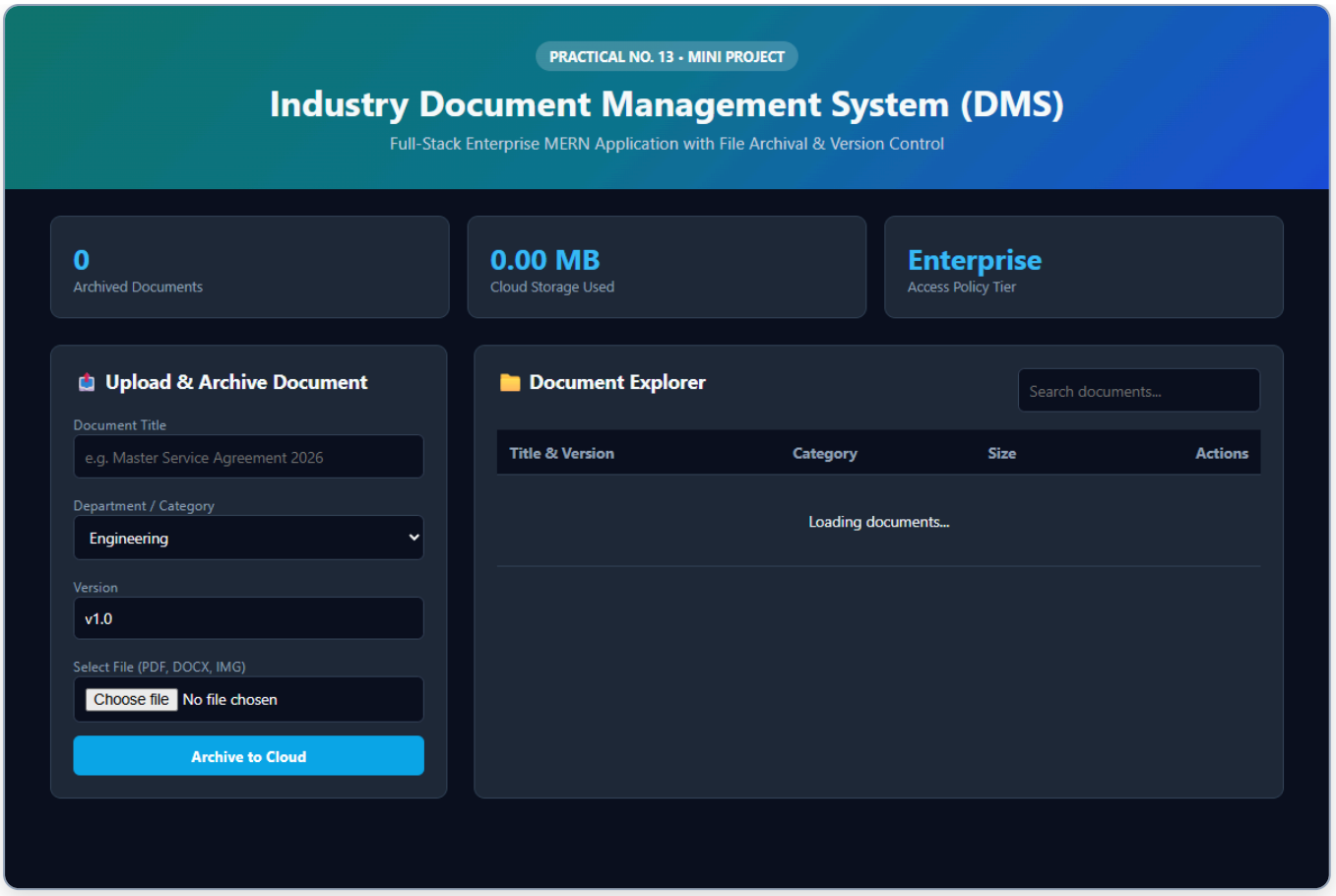
app.get('/api/docs/download/:id', async (req, res) => {
  const doc = await Document.findById(req.params.id);
  res.download(doc.storagePath, doc.originalName);
});

app.delete('/api/docs/:id', async (req, res) => {
  const doc = await Document.findByIdAndDelete(req.params.id);
  if (fs.existsSync(doc.storagePath)) fs.unlinkSync(doc.storagePath);
  res.json({ success: true, message: 'Deleted' });
});

app.listen(PORT, () => console.log(`Practical 13 (DMS Mini Project) on port ${PORT}`));
```

4. Output :

The Document Management System serves as a complete industry-oriented MERN capstone application with document categorization, automated version stamping, secure cloud storage metrics, full-text search, and file retrieval endpoints.



End of Practical Journal

🎓 Ganpat University • Department of Computer Science • M.SC(CA & IT) SEM – 1
MERN Stack Development Practical Documentation • Prince Prajapati (Enrolment No: 26084231038)
End of Practical Journal